ENTERPRISE HRMS

PostgreSQL Database Definition Language (DDL)

Formatted technical reference

Source	User-provided Enterprise HRMS SQL text
Database target	PostgreSQL 14+ (source notes tested logic targets 15/16/17)
Organization	Pre-flight checks, platform layer, and numbered HRMS modules
Handling	SQL text retained; long lines are visually wrapped for readability

Important: This document formats the supplied SQL; it does not certify that the script is safe to execute against a production database. Review assumptions, existing schema, constraints, permissions, and migration order before running it.

Document outline

The source is arranged in the following major sections. The SQL follows in source order.

• PART 0 : PRE-FLIGHT (read-only checks, run and fix before anything else)

• MODULE 00 : PLATFORM LAYER (approvals, numbering, config, audit)

• MODULE 01 : CORE EMPLOYEE INFORMATION & DIRECTORY

• MODULE 02 : ATTENDANCE, BIOMETRIC & GEOFENCING

• MODULE 03 : LEAVE & TIME-OFF

• MODULE 04 : SHIFT PLANNING, ROSTERS, WFH & SWAPS

• MODULE 05 : PAYROLL, STATUTORY COMPLIANCE & PAYSLIPS

• MODULE 06 : RECRUITMENT & APPLICANT TRACKING

• MODULE 07 : LIFECYCLE, ONBOARDING, OFFBOARDING & ASSETS

• MODULE 08 : PERFORMANCE APPRAISAL & OKR/KPI

• MODULE 09 : LEARNING, TRAINING & CERTIFICATIONS

• MODULE 10 : EXPENSE CLAIMS & TRAVEL REIMBURSEMENT

• MODULE 11 : HR HELPDESK, GRIEVANCES & TICKETING

• MODULE 12 : EMPLOYEE ENGAGEMENT, POLLS, SURVEYS & RECOGNITION

• MODULE 13 : EXECUTIVE HR ANALYTICS (10 views, no tables)

• FINAL : updated_at triggers, row-level security, grants, verification

```
-- ====================================================================
-- ENTERPRISE HRMS | Product_DB.public | Module-by-module DDL
-- Source: Enterprise HRMS Master Guide (October 2026)
-- Result: 16 new tables, 12 altered tables, 10 views, helper functions
-- Target: PostgreSQL 14+ (tested logic targets 15/16/17)
-- Run order: PART 0 -> MODULE 00 -> 01 -> 02 -> ... -> 13 -> FINAL
-- Each module is its own transaction. ALTER ... ADD COLUMN uses IF NOT EXISTS;
-- ADD CONSTRAINT / CREATE TRIGGER do not, so run each module once.
-- ====================================================================
--
-- ASSUMED PRIMARY KEYS OF EXISTING TABLES (change if your live DB differs):
--   tab_employee_master.employee_id        integer
--   tab_branch_master.branch_id            integer
--   tab_department_master.department_id    integer
--   tab_employee_role_master.staff_role_id integer
--   tab_common_master.master_id            integer   <-- CONFIRM THIS ONE
--   tab_shift_template.shift_id            uuid
--   tab_employee_attendance.attendance_id  uuid
--   tab_employee_leave_request.leave_id    uuid
--   tab_employee_payroll.payroll_id        uuid
--   tab_announcements / tab_announcement_recipients: no FKs created to them
--
-- ====================================================================
-- PART 0 : PRE-FLIGHT (read-only checks, run and fix before anything else)
-- ====================================================================
/*
-- 0.1 Confirm tables and columns exist
SELECT table_name, column_name, data_type
FROM information_schema.columns
WHERE table_schema='public' AND table_name LIKE 'tab_%'
ORDER BY table_name, ordinal_position;

-- 0.2 Duplicates that would make the new UNIQUE constraints fail (all must return 0 rows)
SELECT org_id, app_id, employee_code, count(*) FROM tab_employee_master GROUP BY 1,2,3 HAVING count(*)>1;
SELECT org_id, employee_id, attendance_date, count(*) FROM tab_employee_attendance GROUP BY 1,2,3 HAVING count(*)>1;
SELECT org_id, branch_id, holiday_date, count(*) FROM tab_branch_holidays GROUP BY 1,2,3 HAVING count(*)>1;

-- 0.3 Is tab_common_master shared with another product? If yes, see NOTE in Module 00.
SELECT DISTINCT master_group FROM tab_common_master ORDER BY 1;
*/

BEGIN;
CREATE EXTENSION IF NOT EXISTS pgcrypto;     -- column encryption (PAN, bank account)
CREATE EXTENSION IF NOT EXISTS btree_gist;   -- exclusion constraints (salary overlap)

-- Generic trigger functions used by many tables ------------------------
CREATE OR REPLACE FUNCTION fn_set_updated_at() RETURNS trigger
LANGUAGE plpgsql AS $$
BEGIN
  NEW.updated_at := now();
  RETURN NEW;
END $$;

CREATE OR REPLACE FUNCTION fn_block_update_delete() RETURNS trigger
LANGUAGE plpgsql AS $$
BEGIN
  RAISE EXCEPTION '% is append-only: % is not allowed (insert a reversing row instead)', TG_TABLE_NAME, TG_OP;
END $$;

-- Encryption helpers. The key is supplied per session by the API:
--   SET app.enc_key = '<key from your vault>';
```

```
CREATE OR REPLACE FUNCTION fn_encrypt(p_text text) RETURNS bytea
LANGUAGE sql AS $$
  SELECT CASE WHEN p_text IS NULL THEN NULL
         ELSE pgp_sym_encrypt(p_text, current_setting('app.enc_key')) END
$$;

CREATE OR REPLACE FUNCTION fn_decrypt(p_data bytea) RETURNS text
LANGUAGE sql AS $$
  SELECT CASE WHEN p_data IS NULL THEN NULL
         ELSE pgp_sym_decrypt(p_data, current_setting('app.enc_key')) END
$$;

-- Monthly partition helper (used for punch log and audit log)
CREATE OR REPLACE FUNCTION fn_create_month_partition(p_parent text, p_month date) RETURNS void
LANGUAGE plpgsql AS $$
DECLARE
  v_from date := date_trunc('month', p_month)::date;
  v_to   date := (date_trunc('month', p_month) + interval '1 month')::date;
  v_name text := p_parent || '_' || to_char(v_from, 'YYYYMM');
BEGIN
  EXECUTE format('CREATE TABLE IF NOT EXISTS %I PARTITION OF %I FOR VALUES FROM (%L) TO (%L)',
                 v_name, p_parent, v_from, v_to);
END $$;
COMMIT;
```

```
-- ====================================================================
-- MODULE 00 : PLATFORM LAYER (approvals, numbering, config, audit)
-- Alters : tab_common_master
-- New    : tab_hrms_request, tab_hrms_approval_step, tab_hrms_audit_log
-- NOTE   : if tab_common_master is shared with another product, create a copy
--          named tab_hrms_config with the same columns and replace the name below
--          (17 new tables, 11 altered tables).
-- ====================================================================
BEGIN;

ALTER TABLE tab_common_master
  ADD COLUMN IF NOT EXISTS parent_id      integer REFERENCES tab_common_master(master_id),
  ADD COLUMN IF NOT EXISTS effective_from date    NOT NULL DEFAULT DATE '2000-01-01',
  ADD COLUMN IF NOT EXISTS effective_to   date,
  ADD COLUMN IF NOT EXISTS rules          jsonb   NOT NULL DEFAULT '{}'::jsonb,
  ADD COLUMN IF NOT EXISTS sort_order     integer NOT NULL DEFAULT 0,
  ADD COLUMN IF NOT EXISTS is_active      boolean NOT NULL DEFAULT true;

ALTER TABLE tab_common_master
  ADD CONSTRAINT uq_common_master UNIQUE (org_id, app_id, master_group, master_code, effective_from),
  ADD CONSTRAINT ck_common_master_dates CHECK (effective_to IS NULL OR effective_to >= effective_from),
  ADD CONSTRAINT ck_common_master_hrms_group CHECK (
    master_group NOT LIKE 'HRMS\_%' OR master_group IN (
      'HRMS_LEAVE_TYPE','HRMS_SALARY_COMP','HRMS_STAT_PF','HRMS_STAT_ESI','HRMS_STAT_PT',
      'HRMS_STAT_LWF','HRMS_TAX_SLAB','HRMS_PREFIX','HRMS_APPROVAL_FLOW','HRMS_CUSTOM_FIELD',
      'HRMS_EXPENSE_CAT','HRMS_SLA','HRMS_TICKET_CAT','HRMS_ASSET_CAT','HRMS_DOC_TYPE',
      'HRMS_DOC_TEMPLATE','HRMS_PERF_CYCLE','HRMS_COURSE','HRMS_SURVEY','HRMS_BADGE',
      'HRMS_DEVICE','HRMS_GRADE','HRMS_COST_CENTER','HRMS_JOB_STAGE','HRMS_NOTIF_TEMPLATE')
  ) NOT VALID;   -- NOT VALID so rows of other products are not re-checked; only new/changed rows

CREATE INDEX IF NOT EXISTS ix_common_master_group
  ON tab_common_master (org_id, app_id, master_group, is_active);

-- Document numbering: EMP-2026-0001, HR-TKT-0001 ... (row lock prevents clashes)
-- Needs an HRMS_PREFIX row, e.g. master_code 'EXPENSE', rules:
--   {"prefix":"EXP-","pattern":"{SEQ:4}","reset":"NEVER","next_seq":1}
--   {"prefix":"EMP-","pattern":"{YYYY}-{SEQ:4}","reset":"YEARLY","next_seq":1}
CREATE OR REPLACE FUNCTION fn_next_doc_no(p_org integer, p_app integer, p_code text)
RETURNS text LANGUAGE plpgsql AS $$
DECLARE
  v_id      integer;
  v_rules   jsonb;
  v_year    text := to_char(now(), 'YYYY');
  v_seq     integer;
  v_pattern text;
  v_width   integer;
BEGIN
  SELECT master_id, rules INTO v_id, v_rules
  FROM tab_common_master
  WHERE org_id = p_org AND app_id = p_app
    AND master_group = 'HRMS_PREFIX' AND master_code = p_code AND is_active
  ORDER BY effective_from DESC
  LIMIT 1
  FOR UPDATE;

  IF v_id IS NULL THEN
    RAISE EXCEPTION 'No active HRMS_PREFIX row for code %', p_code;
  END IF;

  v_seq := COALESCE((v_rules->>'next_seq')::int, 1);
  IF v_rules->>'reset' = 'YEARLY' AND COALESCE(v_rules->>'seq_year','') <> v_year THEN
```

```
    v_seq := 1;
  END IF;

  UPDATE tab_common_master
     SET rules = v_rules || jsonb_build_object('next_seq', v_seq + 1, 'seq_year', v_year)
   WHERE master_id = v_id;

  v_pattern := replace(COALESCE(v_rules->>'pattern', '{SEQ:4}'), '{YYYY}', v_year);
  v_width   := COALESCE(substring(v_pattern from '\{SEQ:(\d+)\}')::int, 4);
  v_pattern := regexp_replace(v_pattern, '\{SEQ:\d+\}', lpad(v_seq::text, v_width, '0'));
  RETURN COALESCE(v_rules->>'prefix', '') || v_pattern;
END $$;

-- ------------------------------------------------------------------
-- D2. tab_hrms_request
-- ------------------------------------------------------------------
CREATE TABLE tab_hrms_request (
  request_id          bigint GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  org_id              integer      NOT NULL,
  app_id              integer      NOT NULL,
  request_no          varchar(30)  NOT NULL,
  request_type        varchar(30)  NOT NULL,
```

```
  employee_id         integer      NOT NULL REFERENCES tab_employee_master(employee_id),
  related_employee_id integer      REFERENCES tab_employee_master(employee_id),
  from_date           date,
  to_date             date,
  effective_date      date,
  amount              numeric(15,2),
  approved_amount     numeric(15,2),
  currency            varchar(3)   NOT NULL DEFAULT 'INR',
  payload             jsonb        NOT NULL DEFAULT '{}'::jsonb,
  old_value           jsonb,
  new_value           jsonb,
  reason              text,
  status              varchar(20)  NOT NULL DEFAULT 'DRAFT',
  current_level       smallint     NOT NULL DEFAULT 1,
  total_levels        smallint     NOT NULL DEFAULT 1,
  payroll_id          uuid         REFERENCES tab_employee_payroll(payroll_id),
  fiscal_year_id      integer,
  submitted_at        timestamptz,
  decided_at          timestamptz,
  created_by          integer,
  created_at          timestamptz  NOT NULL DEFAULT now(),
  updated_at          timestamptz  NOT NULL DEFAULT now(),
  CONSTRAINT uq_hrms_request_no UNIQUE (org_id, app_id, request_no),
  CONSTRAINT ck_request_type CHECK (request_type IN
    ('WFH','SHIFT_SWAP','REGULARIZATION','OVERTIME','COMP_OFF','PROBATION_CONFIRM','PROMOTION',
     'TRANSFER','SALARY_REVISION','RESIGNATION','TERMINATION','LOAN_ADVANCE','JOB_REQUISITION',
     'OFFER','DOC_REQUEST','EXPENSE_CLAIM')),
  CONSTRAINT ck_request_status CHECK (status IN
    ('DRAFT','SUBMITTED','PENDING','APPROVED','REJECTED','CANCELLED','APPLIED','PAID')),
  CONSTRAINT ck_request_levels CHECK (current_level >= 1 AND total_levels >= 1 AND current_level <= total_levels + 1),
  CONSTRAINT ck_request_dates  CHECK (to_date IS NULL OR from_date IS NULL OR to_date >= from_date),
  CONSTRAINT ck_request_amounts CHECK (approved_amount IS NULL OR amount IS NULL OR approved_amount <= amount),
  CONSTRAINT ck_request_payload_obj CHECK (jsonb_typeof(payload) = 'object'),
  -- required keys per type (enforced once the request leaves DRAFT)
  CONSTRAINT ck_request_required CHECK (
    status = 'DRAFT' OR
    CASE
      WHEN request_type = 'WFH'            THEN from_date IS NOT NULL AND to_date IS NOT NULL
      WHEN request_type = 'SHIFT_SWAP'     THEN related_employee_id IS NOT NULL AND from_date IS NOT NULL
      WHEN request_type = 'REGULARIZATION' THEN from_date IS NOT NULL
      WHEN request_type = 'EXPENSE_CLAIM'  THEN from_date IS NOT NULL AND to_date IS NOT NULL
                                                AND jsonb_typeof(payload->'items') = 'array'
      WHEN request_type IN ('PROMOTION','TRANSFER','SALARY_REVISION')
                                           THEN effective_date IS NOT NULL AND new_value IS NOT NULL
      WHEN request_type IN ('RESIGNATION','TERMINATION')
                                           THEN effective_date IS NOT NULL
      WHEN request_type = 'LOAN_ADVANCE'   THEN amount IS NOT NULL
      ELSE true
    END)
);
CREATE INDEX ix_request_emp_type_status ON tab_hrms_request (org_id, employee_id, request_type, status);
CREATE INDEX ix_request_regularization  ON tab_hrms_request (org_id, employee_id, from_date) WHERE request_type = 'REGULARIZATION';
CREATE INDEX ix_request_wfh             ON tab_hrms_request (org_id, employee_id, from_date, to_date) WHERE request_type = 'WFH';
CREATE INDEX ix_request_expense         ON tab_hrms_request (org_id, employee_id, status) WHERE request_type = 'EXPENSE_CLAIM';
CREATE INDEX ix_request_career          ON tab_hrms_request (org_id, effective_date, status)
  WHERE request_type IN ('PROMOTION','TRANSFER','SALARY_REVISION','RESIGNATION','TERMINATION');
CREATE INDEX ix_request_payload_gin     ON tab_hrms_request USING gin (payload jsonb_path_ops);

-- ---------------------------------------------------------------------
-- D3. tab_hrms_approval_step
-- ---------------------------------------------------------------------
```

```
CREATE TABLE tab_hrms_approval_step (
  step_id           bigint GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  org_id            integer      NOT NULL,
  app_id            integer      NOT NULL,
  entity_type       varchar(30)  NOT NULL,
  entity_id         varchar(40)  NOT NULL,
  level_no          smallint     NOT NULL,
  approver_id       integer      NOT NULL REFERENCES tab_employee_master(employee_id),
  approver_role     varchar(30),
  delegated_from_id integer      REFERENCES tab_employee_master(employee_id),
  status            varchar(15)  NOT NULL DEFAULT 'PENDING',
  acted_at          timestamptz,
  comment           text,
  due_at            timestamptz,
  escalated_to_id   integer      REFERENCES tab_employee_master(employee_id),
  created_at        timestamptz  NOT NULL DEFAULT now(),
  updated_at        timestamptz  NOT NULL DEFAULT now(),
  CONSTRAINT uq_approval_step UNIQUE (org_id, entity_type, entity_id, level_no, approver_id),
  CONSTRAINT ck_step_status CHECK (status IN ('PENDING','APPROVED','REJECTED','SKIPPED','ESCALATED')),
  CONSTRAINT ck_step_level  CHECK (level_no >= 1),
  CONSTRAINT ck_step_role   CHECK (approver_role IS NULL OR approver_role IN
    ('MANAGER','HR','FINANCE','DEPT_HEAD','COLLEAGUE','COMPLIANCE')),
  CONSTRAINT ck_step_reject_comment CHECK (status <> 'REJECTED' OR (comment IS NOT NULL AND length(btrim(comment)) > 0))
);
```

```
CREATE INDEX ix_step_inbox  ON tab_hrms_approval_step (org_id, approver_id, status);
CREATE INDEX ix_step_entity ON tab_hrms_approval_step (org_id, entity_type, entity_id);
CREATE INDEX ix_step_due    ON tab_hrms_approval_step (due_at) WHERE status = 'PENDING';

-- ---------------------------------------------------------------------
-- D4. tab_hrms_audit_log  (append-only, partitioned monthly by changed_at)
-- ---------------------------------------------------------------------
CREATE SEQUENCE seq_hrms_audit_log;
CREATE TABLE tab_hrms_audit_log (
  log_id      bigint       NOT NULL DEFAULT nextval('seq_hrms_audit_log'),
  org_id      integer      NOT NULL,
  app_id      integer      NOT NULL,
  table_name  varchar(63)  NOT NULL,
  row_id      varchar(40)  NOT NULL,
  employee_id integer,
  field_name  varchar(63)  NOT NULL,
  old_value   text,
  new_value   text,
  action      varchar(10)  NOT NULL,
  changed_by  integer,
  changed_at  timestamptz  NOT NULL DEFAULT now(),
  ip_address  varchar(45),
  created_at  timestamptz  NOT NULL DEFAULT now(),
  updated_at  timestamptz  NOT NULL DEFAULT now(),
  PRIMARY KEY (log_id, changed_at),
  CONSTRAINT ck_audit_action CHECK (action IN ('INSERT','UPDATE','DELETE'))
) PARTITION BY RANGE (changed_at);

CREATE TABLE tab_hrms_audit_log_default PARTITION OF tab_hrms_audit_log DEFAULT;
DO $$
DECLARE m date;
BEGIN
  FOR m IN SELECT generate_series(date '2026-10-01', date '2027-09-01', interval '1 month')::date LOOP
    PERFORM fn_create_month_partition('tab_hrms_audit_log', m);
  END LOOP;
END $$;
-- Schedule fn_create_month_partition('tab_hrms_audit_log', <next month>) in a monthly job.

CREATE INDEX ix_audit_row ON tab_hrms_audit_log (org_id, table_name, row_id);
CREATE INDEX ix_audit_emp ON tab_hrms_audit_log (org_id, employee_id, changed_at);

REVOKE UPDATE, DELETE, TRUNCATE ON tab_hrms_audit_log FROM PUBLIC;
CREATE TRIGGER trg_audit_append_only
  BEFORE UPDATE OR DELETE ON tab_hrms_audit_log
  FOR EACH ROW EXECUTE FUNCTION fn_block_update_delete();

-- Generic field-level audit trigger.
-- Usage: CREATE TRIGGER ... EXECUTE FUNCTION fn_audit_row('<pk_column>','col1','col2',...)
-- Session settings expected from the API: app.user_id, app.client_ip
CREATE OR REPLACE FUNCTION fn_audit_row() RETURNS trigger
LANGUAGE plpgsql AS $$
DECLARE
  o jsonb := CASE WHEN TG_OP = 'INSERT' THEN '{}'::jsonb ELSE to_jsonb(OLD) END;
  n jsonb := CASE WHEN TG_OP = 'DELETE' THEN '{}'::jsonb ELSE to_jsonb(NEW) END;
  v_row  text := COALESCE(n->>TG_ARGV[0], o->>TG_ARGV[0]);
  v_org  integer := COALESCE((n->>'org_id')::int, (o->>'org_id')::int);
  v_app  integer := COALESCE((n->>'app_id')::int, (o->>'app_id')::int);
  v_emp  integer := COALESCE((n->>'employee_id')::int, (o->>'employee_id')::int,
                             (n->>'assigned_employee_id')::int, (o->>'assigned_employee_id')::int);
  v_user integer := NULLIF(current_setting('app.user_id', true), '')::int;
  v_ip   text    := NULLIF(current_setting('app.client_ip', true), '');
```

```
  col    text;
  i      integer;
BEGIN
  FOR i IN 1 .. TG_NARGS - 1 LOOP
    col := TG_ARGV[i];
    IF (o->>col) IS DISTINCT FROM (n->>col) THEN
      INSERT INTO tab_hrms_audit_log
        (org_id, app_id, table_name, row_id, employee_id, field_name, old_value, new_value,
         action, changed_by, ip_address)
      VALUES
        (v_org, v_app, TG_TABLE_NAME, v_row, v_emp, col,
         CASE WHEN col LIKE '%\_enc' THEN CASE WHEN o->>col IS NULL THEN NULL ELSE '***' END ELSE o->>col END,
         CASE WHEN col LIKE '%\_enc' THEN CASE WHEN n->>col IS NULL THEN NULL ELSE '***' END ELSE n->>col END,
         TG_OP, v_user, v_ip);
    END IF;
  END LOOP;
  RETURN COALESCE(NEW, OLD);
END $$;

COMMIT;


-- ======================================================================
-- MODULE 01 : CORE EMPLOYEE INFORMATION & DIRECTORY
```

```
-- Alters : tab_employee_master, tab_department_master,
--          tab_employee_role_master, tab_branch_master
-- New    : tab_employee_statutory, tab_employee_record
-- ======================================================================
BEGIN;

ALTER TABLE tab_branch_master
  ADD COLUMN IF NOT EXISTS state_code        varchar(5),
  ADD COLUMN IF NOT EXISTS latitude          numeric(9,6),
  ADD COLUMN IF NOT EXISTS longitude         numeric(9,6),
  ADD COLUMN IF NOT EXISTS geofence_radius_m integer;
ALTER TABLE tab_branch_master
  ADD CONSTRAINT ck_branch_geo CHECK (
    (latitude  IS NULL OR latitude  BETWEEN -90  AND 90) AND
    (longitude IS NULL OR longitude BETWEEN -180 AND 180) AND
    (geofence_radius_m IS NULL OR geofence_radius_m > 0));

ALTER TABLE tab_employee_role_master
  ADD COLUMN IF NOT EXISTS grade_code varchar(20),
  ADD COLUMN IF NOT EXISTS level_no   integer;
ALTER TABLE tab_employee_role_master ALTER COLUMN branch_id     DROP NOT NULL;
ALTER TABLE tab_employee_role_master ALTER COLUMN department_id DROP NOT NULL;

ALTER TABLE tab_employee_master
  ADD COLUMN IF NOT EXISTS manager_id          integer REFERENCES tab_employee_master(employee_id),
  ADD COLUMN IF NOT EXISTS grade_code          varchar(20),
  ADD COLUMN IF NOT EXISTS cost_center_code    varchar(20),
  ADD COLUMN IF NOT EXISTS blood_group         varchar(5),
  ADD COLUMN IF NOT EXISTS marital_status      varchar(15),
  ADD COLUMN IF NOT EXISTS nationality         varchar(50),
  ADD COLUMN IF NOT EXISTS personal_email      varchar(150),
  ADD COLUMN IF NOT EXISTS alternate_phone     varchar(30),
  ADD COLUMN IF NOT EXISTS permanent_address   text,
  ADD COLUMN IF NOT EXISTS probation_end_date  date,
  ADD COLUMN IF NOT EXISTS confirmation_date   date,
  ADD COLUMN IF NOT EXISTS resignation_date    date,
  ADD COLUMN IF NOT EXISTS last_working_date   date,
  ADD COLUMN IF NOT EXISTS notice_period_days  integer NOT NULL DEFAULT 30,
  ADD COLUMN IF NOT EXISTS default_shift_id    uuid REFERENCES tab_shift_template(shift_id),
  ADD COLUMN IF NOT EXISTS custom_fields       jsonb NOT NULL DEFAULT '{}'::jsonb;

ALTER TABLE tab_employee_master
  ADD CONSTRAINT uq_employee_code UNIQUE (org_id, app_id, employee_code),
  ADD CONSTRAINT ck_employee_status CHECK (status IN
    ('ACTIVE','PROBATION','NOTICE','TERMINATED','ABSCONDED')) NOT VALID,
  ADD CONSTRAINT ck_employee_not_own_manager CHECK (manager_id IS NULL OR manager_id <> employee_id),
  ADD CONSTRAINT ck_employee_notice CHECK (notice_period_days >= 0);
-- After cleaning legacy status values run:
--   ALTER TABLE tab_employee_master VALIDATE CONSTRAINT ck_employee_status;

CREATE INDEX IF NOT EXISTS ix_employee_manager ON tab_employee_master (org_id, manager_id);
CREATE INDEX IF NOT EXISTS ix_employee_dept    ON tab_employee_master (org_id, app_id, department_id, status);
CREATE INDEX IF NOT EXISTS ix_employee_branch  ON tab_employee_master (org_id, app_id, branch_id);

ALTER TABLE tab_department_master
  ADD COLUMN IF NOT EXISTS parent_department_id integer REFERENCES tab_department_master(department_id),
  ADD COLUMN IF NOT EXISTS head_employee_id     integer REFERENCES tab_employee_master(employee_id);
ALTER TABLE tab_department_master
  ADD CONSTRAINT ck_dept_not_own_parent CHECK (parent_department_id IS NULL OR parent_department_id <> department_id);

-- Block self-reference and circular reporting loops on manager_id
```

```
CREATE OR REPLACE FUNCTION fn_employee_manager_guard() RETURNS trigger
LANGUAGE plpgsql AS $$
BEGIN
  IF NEW.manager_id IS NULL THEN RETURN NEW; END IF;
  IF NEW.manager_id = NEW.employee_id THEN
    RAISE EXCEPTION 'An employee cannot report to themselves';
  END IF;
  IF EXISTS (
    WITH RECURSIVE chain(employee_id, manager_id) AS (
      SELECT employee_id, manager_id FROM tab_employee_master WHERE employee_id = NEW.manager_id
      UNION
      SELECT e.employee_id, e.manager_id
      FROM tab_employee_master e JOIN chain c ON e.employee_id = c.manager_id
    )
    SELECT 1 FROM chain WHERE manager_id = NEW.employee_id
  ) THEN
    RAISE EXCEPTION 'Circular reporting line detected for employee %', NEW.employee_id;
  END IF;
  RETURN NEW;
END $$;

CREATE TRIGGER trg_employee_manager_guard
  BEFORE INSERT OR UPDATE OF manager_id ON tab_employee_master
  FOR EACH ROW EXECUTE FUNCTION fn_employee_manager_guard();
```

```
CREATE TRIGGER trg_employee_audit
  AFTER UPDATE ON tab_employee_master
  FOR EACH ROW EXECUTE FUNCTION fn_audit_row
    ('employee_id','manager_id','staff_role_id','status','department_id','branch_id',
     'grade_code','resignation_date','last_working_date');

-- ------------------------------------------------------------------------
-- D5. tab_employee_statutory  (restricted: HR and payroll roles only)
-- ------------------------------------------------------------------------
CREATE TABLE tab_employee_statutory (
  statutory_id    integer GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  org_id          integer      NOT NULL,
  app_id          integer      NOT NULL,
  employee_id     integer      NOT NULL REFERENCES tab_employee_master(employee_id),
  pan_no_enc      bytea,
  aadhaar_last4   char(4),
  uan_no          varchar(20),
  pf_no           varchar(30),
  esi_no          varchar(20),
  pf_applicable   boolean      NOT NULL DEFAULT true,
  eps_member      boolean      NOT NULL DEFAULT true,
  esi_applicable  boolean      NOT NULL DEFAULT false,
  pt_state_code   varchar(5),
  tax_regime      varchar(5)   NOT NULL DEFAULT 'NEW',
  bank_name       varchar(100),
  bank_branch     varchar(100),
  account_no_enc  bytea,
  ifsc_code       varchar(11),
  account_type    varchar(15),
  kyc_verified_at timestamptz,
  created_at      timestamptz  NOT NULL DEFAULT now(),
  updated_at      timestamptz  NOT NULL DEFAULT now(),
  CONSTRAINT uq_statutory_employee UNIQUE (org_id, employee_id),
  CONSTRAINT ck_statutory_regime   CHECK (tax_regime IN ('OLD','NEW')),
  CONSTRAINT ck_statutory_acct     CHECK (account_type IS NULL OR account_type IN ('SAVINGS','CURRENT','SALARY')),
  CONSTRAINT ck_statutory_aadhaar  CHECK (aadhaar_last4 IS NULL OR aadhaar_last4 ~ '^[0-9]{4}$'),
  CONSTRAINT ck_statutory_ifsc     CHECK (ifsc_code IS NULL OR ifsc_code ~ '^[A-Z]{4}0[A-Z0-9]{6}$')
);
CREATE INDEX ix_statutory_uan ON tab_employee_statutory (org_id, uan_no) WHERE uan_no IS NOT NULL;

CREATE TRIGGER trg_statutory_audit
  AFTER INSERT OR UPDATE ON tab_employee_statutory
  FOR EACH ROW EXECUTE FUNCTION fn_audit_row
    ('statutory_id','pan_no_enc','account_no_enc','ifsc_code','bank_name','uan_no',
     'pf_applicable','esi_applicable','tax_regime','pt_state_code');

-- ------------------------------------------------------------------------
-- D6. tab_employee_record  (documents, family, education, training, tasks ...)
-- ------------------------------------------------------------------------
CREATE TABLE tab_employee_record (
  record_id        bigint GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  org_id           integer      NOT NULL,
  app_id           integer      NOT NULL,
  employee_id      integer      NOT NULL REFERENCES tab_employee_master(employee_id),
  record_type      varchar(25)  NOT NULL,
  title            varchar(200) NOT NULL,
  details          jsonb        NOT NULL DEFAULT '{}'::jsonb,
  file_url         varchar(500),
  issue_date       date,
  expiry_date      date,
  due_date         date,
```

```
  completed_on     date,
  score            numeric(5,2),
  status           varchar(20),
  assigned_to      integer      REFERENCES tab_employee_master(employee_id),
  owner_department varchar(20),
  is_blocking      boolean      NOT NULL DEFAULT false,
  verified_by      integer      REFERENCES tab_employee_master(employee_id),
  verified_at      timestamptz,
  created_at       timestamptz  NOT NULL DEFAULT now(),
  updated_at       timestamptz  NOT NULL DEFAULT now(),
  CONSTRAINT ck_record_type CHECK (record_type IN
    ('DOCUMENT','ID_PROOF','EMERGENCY_CONTACT','DEPENDENT','NOMINEE','EDUCATION','EXPERIENCE',
     'SKILL','CERTIFICATION','TRAINING','TAX_DECLARATION','POLICY_ACK','ONBOARD_TASK',
     'OFFBOARD_TASK','PROBATION_TASK')),
  CONSTRAINT ck_record_status CHECK (status IS NULL OR status IN
    ('PENDING','VERIFIED','REJECTED','ENROLLED','IN_PROGRESS','COMPLETED','DROPPED','EXPIRED','OPEN','DONE')),
  CONSTRAINT ck_record_owner_dept CHECK (owner_department IS NULL OR owner_department IN
    ('HR','IT','FINANCE','MANAGER')),
  CONSTRAINT ck_record_details CHECK (jsonb_typeof(details) = 'object'),
  CONSTRAINT ck_record_score   CHECK (score IS NULL OR score BETWEEN 0 AND 100)
);
CREATE INDEX ix_record_emp_type    ON tab_employee_record (org_id, employee_id, record_type);
CREATE INDEX ix_record_documents   ON tab_employee_record (org_id, employee_id, expiry_date) WHERE record_type = 'DOCUMENT';
```

```
CREATE INDEX ix_record_training    ON tab_employee_record (org_id, employee_id, status, due_date) WHERE record_type = 'TRAINING';
CREATE INDEX ix_record_onboard     ON tab_employee_record (org_id, employee_id, status) WHERE record_type = 'ONBOARD_TASK';
CREATE INDEX ix_record_offboard    ON tab_employee_record (org_id, employee_id, status) WHERE record_type = 'OFFBOARD_TASK';
CREATE INDEX ix_record_blocking    ON tab_employee_record (org_id, employee_id) WHERE is_blocking AND status IS DISTINCT FROM 'DONE';
CREATE INDEX ix_record_expiry      ON tab_employee_record (expiry_date) WHERE expiry_date IS NOT NULL;
-- stop duplicate training assignment
CREATE UNIQUE INDEX uq_record_training ON tab_employee_record (employee_id, record_type, title, due_date)
  WHERE record_type = 'TRAINING';

COMMIT;


-- ====================================================================
-- MODULE 02 : ATTENDANCE, BIOMETRIC & GEOFENCING
-- Alters : tab_employee_attendance
-- New    : tab_attendance_punch
-- ====================================================================
BEGIN;

-- Convert check-in/out to timestamptz only if they are still "timestamp without time zone"
DO $$
DECLARE c text;
BEGIN
  FOREACH c IN ARRAY ARRAY['check_in_time','check_out_time'] LOOP
    IF EXISTS (SELECT 1 FROM information_schema.columns
               WHERE table_schema='public' AND table_name='tab_employee_attendance'
                 AND column_name=c AND data_type='timestamp without time zone') THEN
      EXECUTE format('ALTER TABLE tab_employee_attendance ALTER COLUMN %I TYPE timestamptz USING %I AT TIME ZONE ''Asia/Kolkata''', c, c);
    END IF;
  END LOOP;
END $$;

ALTER TABLE tab_employee_attendance
  ADD COLUMN IF NOT EXISTS shift_id                  uuid REFERENCES tab_shift_template(shift_id),
  ADD COLUMN IF NOT EXISTS shift_date                date,
  ADD COLUMN IF NOT EXISTS in_latitude               numeric(9,6),
  ADD COLUMN IF NOT EXISTS in_longitude              numeric(9,6),
  ADD COLUMN IF NOT EXISTS out_latitude              numeric(9,6),
  ADD COLUMN IF NOT EXISTS out_longitude             numeric(9,6),
  ADD COLUMN IF NOT EXISTS check_in_device           varchar(20),
  ADD COLUMN IF NOT EXISTS check_out_device          varchar(20),
  ADD COLUMN IF NOT EXISTS in_ip                     varchar(45),
  ADD COLUMN IF NOT EXISTS out_ip                    varchar(45),
  ADD COLUMN IF NOT EXISTS late_minutes              integer NOT NULL DEFAULT 0,
  ADD COLUMN IF NOT EXISTS early_exit_minutes        integer NOT NULL DEFAULT 0,
  ADD COLUMN IF NOT EXISTS break_minutes             integer NOT NULL DEFAULT 0,
  ADD COLUMN IF NOT EXISTS overtime_minutes          integer NOT NULL DEFAULT 0,
  ADD COLUMN IF NOT EXISTS is_regularized            boolean NOT NULL DEFAULT false,
  ADD COLUMN IF NOT EXISTS regularization_reason     text,
  ADD COLUMN IF NOT EXISTS regularization_request_id  bigint REFERENCES tab_hrms_request(request_id);

ALTER TABLE tab_employee_attendance
  ADD CONSTRAINT uq_attendance_emp_date UNIQUE (org_id, employee_id, attendance_date),
  ADD CONSTRAINT ck_attendance_status CHECK (status IN
    ('PRESENT','ABSENT','HALF_DAY','LATE','ON_LEAVE','HOLIDAY','WEEK_OFF','WFH')) NOT VALID,
  ADD CONSTRAINT ck_attendance_minutes CHECK
    (late_minutes >= 0 AND early_exit_minutes >= 0 AND break_minutes >= 0 AND overtime_minutes >= 0);
-- After cleaning legacy status values:
--   ALTER TABLE tab_employee_attendance VALIDATE CONSTRAINT ck_attendance_status;

CREATE INDEX IF NOT EXISTS ix_attendance_org_date ON tab_employee_attendance (org_id, app_id, attendance_date);
```

```
CREATE INDEX IF NOT EXISTS ix_attendance_emp_date ON tab_employee_attendance (org_id, employee_id, attendance_date);

-- ---------------------------------------------------------------------
-- D7. tab_attendance_punch  (raw log, partitioned monthly by punch_time)
-- ---------------------------------------------------------------------
CREATE SEQUENCE seq_attendance_punch;
CREATE TABLE tab_attendance_punch (
  punch_id       bigint       NOT NULL DEFAULT nextval('seq_attendance_punch'),
  org_id         integer      NOT NULL,
  app_id         integer      NOT NULL,
  employee_id    integer      NOT NULL REFERENCES tab_employee_master(employee_id),
  punch_time     timestamptz  NOT NULL,
  direction      varchar(3)   NOT NULL,
  source         varchar(12)  NOT NULL,
  device_code    varchar(50),
  latitude       numeric(9,6),
  longitude      numeric(9,6),
  ip_address     varchar(45),
  is_valid       boolean      NOT NULL DEFAULT true,
  invalid_reason varchar(100),
  attendance_id  uuid         REFERENCES tab_employee_attendance(attendance_id),
  created_at     timestamptz  NOT NULL DEFAULT now(),
  updated_at     timestamptz  NOT NULL DEFAULT now(),
  PRIMARY KEY (punch_id, punch_time),
```

```
  CONSTRAINT ck_punch_direction CHECK (direction IN ('IN','OUT')),
  CONSTRAINT ck_punch_source    CHECK (source IN ('BIOMETRIC','WEB','MOBILE')),
  CONSTRAINT ck_punch_reason    CHECK (is_valid OR invalid_reason IS NOT NULL),
  CONSTRAINT ck_punch_geo CHECK (
    (latitude  IS NULL OR latitude  BETWEEN -90  AND 90) AND
    (longitude IS NULL OR longitude BETWEEN -180 AND 180))
) PARTITION BY RANGE (punch_time);

CREATE TABLE tab_attendance_punch_default PARTITION OF tab_attendance_punch DEFAULT;
DO $$
DECLARE m date;
BEGIN
  FOR m IN SELECT generate_series(date '2026-10-01', date '2027-09-01', interval '1 month')::date LOOP
    PERFORM fn_create_month_partition('tab_attendance_punch', m);
  END LOOP;
END $$;
-- Schedule fn_create_month_partition('tab_attendance_punch', <next month>) in a monthly job.

-- stops duplicate device uploads (unique index on a partitioned table must include punch_time)
CREATE UNIQUE INDEX uq_punch_device ON tab_attendance_punch (org_id, employee_id, punch_time, device_code);
CREATE INDEX ix_punch_emp_time ON tab_attendance_punch (org_id, employee_id, punch_time);
CREATE INDEX ix_punch_invalid  ON tab_attendance_punch (org_id, punch_time) WHERE NOT is_valid;

COMMIT;


-- ====================================================================
-- MODULE 03 : LEAVE & TIME-OFF
-- Alters : tab_employee_leave_request, tab_branch_holidays
-- New    : tab_leave_ledger  (+ view v_leave_balance)
-- ====================================================================
BEGIN;

ALTER TABLE tab_employee_leave_request
  ADD COLUMN IF NOT EXISTS leave_type_id             integer REFERENCES tab_common_master(master_id),
  ADD COLUMN IF NOT EXISTS is_half_day               boolean NOT NULL DEFAULT false,
  ADD COLUMN IF NOT EXISTS half_day_period           varchar(10),
  ADD COLUMN IF NOT EXISTS document_proof_url        varchar(500),
  ADD COLUMN IF NOT EXISTS rejection_reason          text,
  ADD COLUMN IF NOT EXISTS approved_by_employee_id   integer REFERENCES tab_employee_master(employee_id),
  ADD COLUMN IF NOT EXISTS current_level             smallint NOT NULL DEFAULT 1,
  ADD COLUMN IF NOT EXISTS fiscal_year_id            integer;
-- legacy columns leave_type (text) and approved_by (uuid) stay untouched for old rows

ALTER TABLE tab_employee_leave_request
  ADD CONSTRAINT ck_leave_status CHECK (status IN
    ('DRAFT','SUBMITTED','PENDING','APPROVED','REJECTED','CANCELLED')) NOT VALID,
  ADD CONSTRAINT ck_leave_half_day CHECK (
    (is_half_day AND half_day_period IN ('FIRST','SECOND')) OR (NOT is_half_day AND half_day_period IS NULL)) NOT VALID,
  ADD CONSTRAINT ck_leave_dates CHECK (end_date >= start_date),
  ADD CONSTRAINT ck_leave_reject_reason CHECK (status <> 'REJECTED' OR rejection_reason IS NOT NULL) NOT VALID;

CREATE INDEX IF NOT EXISTS ix_leave_emp_dates ON tab_employee_leave_request (org_id, employee_id, start_date, end_date);
CREATE INDEX IF NOT EXISTS ix_leave_pending   ON tab_employee_leave_request (org_id, status) WHERE status IN ('SUBMITTED','PENDING');

ALTER TABLE tab_branch_holidays
  ADD COLUMN IF NOT EXISTS holiday_type             varchar(15) NOT NULL DEFAULT 'PUBLIC',
  ADD COLUMN IF NOT EXISTS is_optional_quota_counted boolean    NOT NULL DEFAULT false;
ALTER TABLE tab_branch_holidays
  ADD CONSTRAINT uq_holiday_branch_date UNIQUE (org_id, branch_id, holiday_date),
  ADD CONSTRAINT ck_holiday_type CHECK (holiday_type IN ('PUBLIC','OPTIONAL','RESTRICTED'));
```

```
-- Block overlapping approved leave for the same employee
CREATE OR REPLACE FUNCTION fn_leave_overlap_guard() RETURNS trigger
LANGUAGE plpgsql AS $$
BEGIN
  IF NEW.status = 'APPROVED' AND EXISTS (
    SELECT 1 FROM tab_employee_leave_request l
    WHERE l.org_id = NEW.org_id AND l.employee_id = NEW.employee_id
      AND l.status = 'APPROVED' AND l.leave_id <> NEW.leave_id
      AND l.start_date <= NEW.end_date AND l.end_date >= NEW.start_date
      AND NOT (COALESCE(l.is_half_day,false) AND COALESCE(NEW.is_half_day,false)
               AND l.half_day_period IS DISTINCT FROM NEW.half_day_period
               AND l.start_date = NEW.start_date AND l.end_date = NEW.end_date)
  ) THEN
    RAISE EXCEPTION 'Leave overlaps an existing approved leave for employee %', NEW.employee_id;
  END IF;
  RETURN NEW;
END $$;

CREATE TRIGGER trg_leave_overlap
  BEFORE INSERT OR UPDATE OF status, start_date, end_date ON tab_employee_leave_request
  FOR EACH ROW EXECUTE FUNCTION fn_leave_overlap_guard();

-- ---------------------------------------------------------------------
```

```
-- D8. tab_leave_ledger  (append-only)
-- ---------------------------------------------------------------------
CREATE TABLE tab_leave_ledger (
  ledger_id      bigint GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  org_id         integer      NOT NULL,
  app_id         integer      NOT NULL,
  employee_id    integer      NOT NULL REFERENCES tab_employee_master(employee_id),
  leave_type_id  integer      NOT NULL REFERENCES tab_common_master(master_id),
  fiscal_year_id integer      NOT NULL,
  entry_type     varchar(12)  NOT NULL,
  entry_date     date         NOT NULL,
  days           numeric(5,2) NOT NULL,
  leave_id       uuid         REFERENCES tab_employee_leave_request(leave_id),
  remarks        varchar(200),
  created_by     integer,
  created_at     timestamptz  NOT NULL DEFAULT now(),
  updated_at     timestamptz  NOT NULL DEFAULT now(),
  CONSTRAINT ck_ledger_entry_type CHECK (entry_type IN
    ('OPENING','ACCRUAL','AVAIL','ADJUST','ENCASH','LAPSE','CARRY_FWD')),
  CONSTRAINT ck_ledger_days_nonzero CHECK (days <> 0),
  CONSTRAINT ck_ledger_adjust_remark CHECK (entry_type <> 'ADJUST' OR remarks IS NOT NULL)
);
CREATE INDEX ix_ledger_balance ON tab_leave_ledger (org_id, employee_id, leave_type_id, fiscal_year_id);
CREATE INDEX ix_ledger_leave   ON tab_leave_ledger (leave_id) WHERE leave_id IS NOT NULL;

REVOKE UPDATE, DELETE ON tab_leave_ledger FROM PUBLIC;
CREATE TRIGGER trg_ledger_append_only
  BEFORE UPDATE OR DELETE ON tab_leave_ledger
  FOR EACH ROW EXECUTE FUNCTION fn_block_update_delete();

-- Balance = sum of ledger days (no balance table that can drift)
CREATE OR REPLACE VIEW v_leave_balance AS
WITH l AS (
  SELECT org_id, app_id, employee_id, leave_type_id, fiscal_year_id,
         COALESCE(SUM(days) FILTER (WHERE entry_type = 'OPENING'),   0) AS opening,
         COALESCE(SUM(days) FILTER (WHERE entry_type = 'ACCRUAL'),   0) AS accrued,
         COALESCE(SUM(days) FILTER (WHERE entry_type = 'CARRY_FWD'), 0) AS carried_forward,
         COALESCE(-SUM(days) FILTER (WHERE entry_type = 'AVAIL'),    0) AS availed,
         COALESCE(SUM(days) FILTER (WHERE entry_type = 'ADJUST'),    0) AS adjusted,
         COALESCE(-SUM(days) FILTER (WHERE entry_type = 'ENCASH'),   0) AS encashed,
         COALESCE(-SUM(days) FILTER (WHERE entry_type = 'LAPSE'),    0) AS lapsed,
         SUM(days) AS current_balance
  FROM tab_leave_ledger
  GROUP BY org_id, app_id, employee_id, leave_type_id, fiscal_year_id
), p AS (
  SELECT org_id, employee_id, leave_type_id, fiscal_year_id, SUM(days_count) AS pending_days
  FROM tab_employee_leave_request
  WHERE status IN ('SUBMITTED','PENDING')
  GROUP BY org_id, employee_id, leave_type_id, fiscal_year_id
)
SELECT l.*, COALESCE(p.pending_days, 0) AS pending_days,
       l.current_balance - COALESCE(p.pending_days, 0) AS available_balance
FROM l
LEFT JOIN p ON p.org_id = l.org_id AND p.employee_id = l.employee_id
           AND p.leave_type_id = l.leave_type_id AND p.fiscal_year_id = l.fiscal_year_id;

COMMIT;


-- ====================================================================
-- MODULE 04 : SHIFT PLANNING, ROSTERS, WFH & SWAPS
```

```
-- Alters : tab_shift_template
-- New    : none (WFH and swaps are tab_hrms_request rows)
-- ========================================================================
BEGIN;

ALTER TABLE tab_shift_template
  ADD COLUMN IF NOT EXISTS grace_late_minutes   integer      NOT NULL DEFAULT 0,
  ADD COLUMN IF NOT EXISTS half_day_min_minutes integer,
  ADD COLUMN IF NOT EXISTS full_day_min_minutes integer,
  ADD COLUMN IF NOT EXISTS is_night_shift       boolean      NOT NULL DEFAULT false,
  ADD COLUMN IF NOT EXISTS weekly_off_days      varchar(30),
  ADD COLUMN IF NOT EXISTS ot_multiplier        numeric(3,2) NOT NULL DEFAULT 1.00;

ALTER TABLE tab_shift_template
  ADD CONSTRAINT ck_shift_grace CHECK (grace_late_minutes >= 0),
  ADD CONSTRAINT ck_shift_thresholds CHECK (
    half_day_min_minutes IS NULL OR full_day_min_minutes IS NULL OR half_day_min_minutes <= full_day_min_minutes),
  ADD CONSTRAINT ck_shift_ot CHECK (ot_multiplier >= 1.00);
-- weekly_off_days example value: 'SUN' or 'SAT,SUN'. Column consult_duration_minutes is ignored.

-- OPTIONAL (run only after cleaning overlapping roster rows; exclusion constraints cannot be NOT VALID):
-- ALTER TABLE tab_employee_shift_assignment
--   ADD CONSTRAINT ex_roster_no_overlap EXCLUDE USING gist (
--     employee_id WITH =, day_of_week WITH =,
```

```
--     daterange(effective_from, effective_to, '[]') WITH &&);

COMMIT;


-- ======================================================================
-- MODULE 05 : PAYROLL, STATUTORY COMPLIANCE & PAYSLIPS
-- Alters : tab_employee_payroll
-- New    : tab_payroll_run, tab_employee_salary, tab_payroll_item_detail
-- ======================================================================
BEGIN;

-- ----------------------------------------------------------------------
-- D9. tab_payroll_run
-- ----------------------------------------------------------------------
CREATE TABLE tab_payroll_run (
  run_id         bigint GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  org_id         integer      NOT NULL,
  app_id         integer      NOT NULL,
  run_type       varchar(15)  NOT NULL,
  run_seq        smallint     NOT NULL DEFAULT 1,
  pay_period     date         NOT NULL,
  cutoff_date    date         NOT NULL,
  day_basis      varchar(10)  NOT NULL DEFAULT 'CALENDAR',
  status         varchar(12)  NOT NULL DEFAULT 'DRAFT',
  employee_count integer,
  total_gross    numeric(15,2),
  total_net      numeric(15,2),
  fiscal_year_id integer,
  computed_at    timestamptz,
  approved_by    integer      REFERENCES tab_employee_master(employee_id),
  approved_at    timestamptz,
  locked_at      timestamptz,
  paid_at        timestamptz,
  created_at     timestamptz  NOT NULL DEFAULT now(),
  updated_at     timestamptz  NOT NULL DEFAULT now(),
  CONSTRAINT uq_payroll_run UNIQUE (org_id, app_id, pay_period, run_type, run_seq),
  CONSTRAINT ck_run_type    CHECK (run_type IN ('REGULAR','SUPPLEMENTARY','BONUS','FNF')),
  CONSTRAINT ck_run_status  CHECK (status IN ('DRAFT','COMPUTED','APPROVED','LOCKED','PAID')),
  CONSTRAINT ck_run_basis   CHECK (day_basis IN ('CALENDAR','26','30')),
  CONSTRAINT ck_run_period  CHECK (pay_period = date_trunc('month', pay_period)::date),
  CONSTRAINT ck_run_locked  CHECK (status NOT IN ('LOCKED','PAID') OR locked_at IS NOT NULL)
);
CREATE INDEX ix_run_period ON tab_payroll_run (org_id, app_id, pay_period, status);

-- ----------------------------------------------------------------------
-- tab_employee_payroll (existing payslip header)
-- ----------------------------------------------------------------------
ALTER TABLE tab_employee_payroll
  ALTER COLUMN basic_salary TYPE numeric(15,2),
  ALTER COLUMN allowances   TYPE numeric(15,2),
  ALTER COLUMN deductions   TYPE numeric(15,2),
  ALTER COLUMN net_salary   TYPE numeric(15,2);

ALTER TABLE tab_employee_payroll
  ADD COLUMN IF NOT EXISTS run_id              bigint REFERENCES tab_payroll_run(run_id),
  ADD COLUMN IF NOT EXISTS pay_period          date,
  ADD COLUMN IF NOT EXISTS payslip_no          varchar(50),
  ADD COLUMN IF NOT EXISTS total_days          numeric(5,2),
  ADD COLUMN IF NOT EXISTS payable_days        numeric(5,2),
  ADD COLUMN IF NOT EXISTS lop_days            numeric(5,2),
```

```
  ADD COLUMN IF NOT EXISTS gross_earnings       numeric(15,2),
  ADD COLUMN IF NOT EXISTS total_deductions     numeric(15,2),
  ADD COLUMN IF NOT EXISTS reimbursements       numeric(15,2) NOT NULL DEFAULT 0,
  ADD COLUMN IF NOT EXISTS net_pay              numeric(15,2),
  ADD COLUMN IF NOT EXISTS employer_contribution numeric(15,2),
  ADD COLUMN IF NOT EXISTS overtime_pay         numeric(15,2) NOT NULL DEFAULT 0,
  ADD COLUMN IF NOT EXISTS payment_method       varchar(30),
  ADD COLUMN IF NOT EXISTS payslip_url          varchar(500),
  ADD COLUMN IF NOT EXISTS status               varchar(12);
-- pay_period_month (text) stays for old rows. Backfill pay_period from it once you know its format.

ALTER TABLE tab_employee_payroll
  ADD CONSTRAINT uq_payroll_emp_period_run UNIQUE (org_id, employee_id, pay_period, run_id),
  ADD CONSTRAINT ck_payroll_status CHECK (status IS NULL OR status IN
    ('DRAFT','COMPUTED','APPROVED','LOCKED','PAID')),
  ADD CONSTRAINT ck_payroll_days CHECK (
    payable_days IS NULL OR total_days IS NULL OR (payable_days >= 0 AND payable_days <= total_days)),
  ADD CONSTRAINT ck_payroll_net CHECK (
    net_pay IS NULL OR gross_earnings IS NULL OR
    net_pay = gross_earnings - COALESCE(total_deductions,0) + COALESCE(reimbursements,0)) NOT VALID;
CREATE INDEX IF NOT EXISTS ix_payroll_run    ON tab_employee_payroll (org_id, run_id);
CREATE INDEX IF NOT EXISTS ix_payroll_period ON tab_employee_payroll (org_id, app_id, pay_period);

-- ---------------------------------------------------------------------
```

```
-- D10. tab_employee_salary  (effective-dated CTC)
-- ------------------------------------------------------------------
CREATE TABLE tab_employee_salary (
  salary_id           bigint GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  org_id              integer       NOT NULL,
  app_id              integer       NOT NULL,
  employee_id         integer       NOT NULL REFERENCES tab_employee_master(employee_id),
  effective_from      date          NOT NULL,
  effective_to        date,
  annual_ctc          numeric(15,2) NOT NULL,
  monthly_gross       numeric(15,2) NOT NULL,
  components          jsonb         NOT NULL,
  revision_request_id bigint        REFERENCES tab_hrms_request(request_id),
  status              varchar(12)   NOT NULL DEFAULT 'ACTIVE',
  created_at          timestamptz   NOT NULL DEFAULT now(),
  updated_at          timestamptz   NOT NULL DEFAULT now(),
  CONSTRAINT ck_salary_status     CHECK (status IN ('DRAFT','ACTIVE','SUPERSEDED')),
  CONSTRAINT ck_salary_dates      CHECK (effective_to IS NULL OR effective_to >= effective_from),
  CONSTRAINT ck_salary_amounts    CHECK (annual_ctc >= 0 AND monthly_gross >= 0),
  CONSTRAINT ck_salary_components CHECK (jsonb_typeof(components) = 'array'),
  -- no overlapping effective ranges per employee (drafts excluded)
  CONSTRAINT ex_salary_no_overlap EXCLUDE USING gist (
    org_id WITH =, employee_id WITH =,
    daterange(effective_from, effective_to, '[]') WITH &&
  ) WHERE (status <> 'DRAFT')
);
CREATE INDEX ix_salary_emp ON tab_employee_salary (org_id, employee_id, effective_from DESC);

CREATE TRIGGER trg_salary_audit
  AFTER INSERT OR UPDATE ON tab_employee_salary
  FOR EACH ROW EXECUTE FUNCTION fn_audit_row
    ('salary_id','annual_ctc','monthly_gross','components','effective_from','effective_to','status');

-- ------------------------------------------------------------------
-- D11. tab_payroll_item_detail
-- ------------------------------------------------------------------
-- Convention used by the statutory views in Module 13:
--   statutory lines carry rate_snapshot->>'head' in
--   PF_EE, PF_EPF, PF_EPS, PF_EDLI, PF_ADMIN, ESI_EE, ESI_ER, PT, LWF, TDS
--   and rate_snapshot->>'wage' = the wage the rate was applied on.
CREATE TABLE tab_payroll_item_detail (
  item_id       bigint GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  org_id        integer       NOT NULL,
  app_id        integer       NOT NULL,
  payroll_id    uuid          REFERENCES tab_employee_payroll(payroll_id),
  run_id        bigint        REFERENCES tab_payroll_run(run_id),
  employee_id   integer       NOT NULL REFERENCES tab_employee_master(employee_id),
  pay_period    date          NOT NULL,
  component_id  integer       REFERENCES tab_common_master(master_id),
  item_type     varchar(18)   NOT NULL,
  item_name     varchar(100)  NOT NULL,
  source        varchar(10)   NOT NULL DEFAULT 'SYSTEM',
  amount        numeric(15,2) NOT NULL,
  rate_snapshot jsonb,
  remarks       varchar(200),
  created_at    timestamptz   NOT NULL DEFAULT now(),
  updated_at    timestamptz   NOT NULL DEFAULT now(),
  CONSTRAINT ck_item_type   CHECK (item_type IN
    ('EARNING','DEDUCTION','EMPLOYER_CONTRIB','REIMBURSEMENT','ADJUSTMENT')),
  CONSTRAINT ck_item_source CHECK (source IN
    ('SYSTEM','MANUAL','LOAN','ARREAR','BONUS','ENCASH','FNF','EXPENSE')),
```

```
  CONSTRAINT ck_item_period CHECK (pay_period = date_trunc('month', pay_period)::date),
  CONSTRAINT ck_item_snapshot CHECK (rate_snapshot IS NULL OR jsonb_typeof(rate_snapshot) = 'object'),
  CONSTRAINT ck_item_sign CHECK (
    item_type = 'ADJUSTMENT' OR amount >= 0)
);
CREATE INDEX ix_item_payroll ON tab_payroll_item_detail (payroll_id);
CREATE INDEX ix_item_emp     ON tab_payroll_item_detail (org_id, employee_id, pay_period);
CREATE INDEX ix_item_run     ON tab_payroll_item_detail (org_id, run_id, item_type);
CREATE INDEX ix_item_head    ON tab_payroll_item_detail (org_id, pay_period, (rate_snapshot->>'head'))
  WHERE rate_snapshot ? 'head';

-- Frozen month guard: once a REGULAR run for a month is LOCKED/PAID, attendance, leave and salary
-- for that month cannot change. Usage: fn_block_locked_month('<date column>')
CREATE OR REPLACE FUNCTION fn_block_locked_month() RETURNS trigger
LANGUAGE plpgsql AS $$
DECLARE
  v_new date := CASE WHEN TG_OP = 'DELETE' THEN NULL ELSE (to_jsonb(NEW)->>TG_ARGV[0])::date END;
  v_old date := CASE WHEN TG_OP = 'INSERT' THEN NULL ELSE (to_jsonb(OLD)->>TG_ARGV[0])::date END;
  v_org integer := COALESCE((to_jsonb(NEW)->>'org_id')::int, (to_jsonb(OLD)->>'org_id')::int);
  v_d   date;
BEGIN
  FOREACH v_d IN ARRAY ARRAY[v_new, v_old] LOOP
    IF v_d IS NOT NULL AND EXISTS (
      SELECT 1 FROM tab_payroll_run r
```

```
      WHERE r.org_id = v_org AND r.run_type = 'REGULAR'
        AND r.pay_period = date_trunc('month', v_d)::date
        AND r.status IN ('LOCKED','PAID')
    ) THEN
      RAISE EXCEPTION 'Payroll for % is locked; % on % is not allowed',
                      to_char(v_d, 'Mon YYYY'), TG_OP, TG_TABLE_NAME;
    END IF;
  END LOOP;
  RETURN COALESCE(NEW, OLD);
END $$;

CREATE TRIGGER trg_lock_attendance
  BEFORE INSERT OR UPDATE OR DELETE ON tab_employee_attendance
  FOR EACH ROW EXECUTE FUNCTION fn_block_locked_month('attendance_date');
CREATE TRIGGER trg_lock_leave
  BEFORE INSERT OR UPDATE OR DELETE ON tab_employee_leave_request
  FOR EACH ROW EXECUTE FUNCTION fn_block_locked_month('start_date');
CREATE TRIGGER trg_lock_salary
  BEFORE INSERT OR UPDATE OR DELETE ON tab_employee_salary
  FOR EACH ROW EXECUTE FUNCTION fn_block_locked_month('effective_from');

COMMIT;


-- ======================================================================
-- MODULE 06 : RECRUITMENT & APPLICANT TRACKING
-- Alters : none
-- New    : tab_job_posting, tab_job_applicant
-- ======================================================================
BEGIN;

-- D12. tab_job_posting
CREATE TABLE tab_job_posting (
  job_id                integer GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  org_id                integer       NOT NULL,
  app_id                integer       NOT NULL,
  job_code              varchar(30)   NOT NULL,
  title                 varchar(150)  NOT NULL,
  department_id         integer       REFERENCES tab_department_master(department_id),
  branch_id             integer       REFERENCES tab_branch_master(branch_id),
  requisition_request_id bigint       REFERENCES tab_hrms_request(request_id),
  job_type              varchar(20)   NOT NULL DEFAULT 'FULL_TIME',
  openings              integer       NOT NULL DEFAULT 1,
  min_experience        numeric(3,1)  NOT NULL DEFAULT 0,
  salary_min            numeric(12,2),
  salary_max            numeric(12,2),
  description           text,
  closing_date          date,
  status                varchar(10)   NOT NULL DEFAULT 'OPEN',
  created_at            timestamptz   NOT NULL DEFAULT now(),
  updated_at            timestamptz   NOT NULL DEFAULT now(),
  CONSTRAINT uq_job_code    UNIQUE (org_id, app_id, job_code),
  CONSTRAINT ck_job_type    CHECK (job_type IN ('FULL_TIME','CONTRACT','INTERN')),
  CONSTRAINT ck_job_status  CHECK (status IN ('OPEN','PAUSED','CLOSED')),
  CONSTRAINT ck_job_openings CHECK (openings >= 1),
  CONSTRAINT ck_job_salary  CHECK (salary_min IS NULL OR salary_max IS NULL OR salary_min <= salary_max)
);
CREATE INDEX ix_job_status ON tab_job_posting (org_id, app_id, status);

-- D13. tab_job_applicant
CREATE TABLE tab_job_applicant (
```

```
applicant_id            integer GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
org_id                  integer       NOT NULL,
app_id                  integer       NOT NULL,
job_id                  integer       NOT NULL REFERENCES tab_job_posting(job_id),
full_name               varchar(150)  NOT NULL,
email                   varchar(150)  NOT NULL,
phone                   varchar(30),
resume_url              varchar(500),
source                  varchar(20),
referred_by_employee_id integer       REFERENCES tab_employee_master(employee_id),
current_ctc             numeric(12,2),
expected_ctc            numeric(12,2),
stage                   varchar(20)   NOT NULL DEFAULT 'APPLIED',
stage_history           jsonb         NOT NULL DEFAULT '[]'::jsonb,
interviews              jsonb         NOT NULL DEFAULT '[]'::jsonb,
offer                   jsonb,
overall_score           numeric(3,1),
converted_employee_id   integer       REFERENCES tab_employee_master(employee_id),
created_at              timestamptz   NOT NULL DEFAULT now(),
updated_at              timestamptz   NOT NULL DEFAULT now(),
CONSTRAINT uq_applicant_job_email UNIQUE (org_id, job_id, email),
CONSTRAINT ck_applicant_stage  CHECK (stage IN
  ('APPLIED','SCREENED','TECHNICAL','HR_INTERVIEW','OFFERED','HIRED','REJECTED')),
CONSTRAINT ck_applicant_source CHECK (source IS NULL OR source IN ('DIRECT','REFERRAL','JOB_BOARD','AGENCY')),
```

```
  CONSTRAINT ck_applicant_history    CHECK (jsonb_typeof(stage_history) = 'array'),
  CONSTRAINT ck_applicant_interviews CHECK (jsonb_typeof(interviews) = 'array'),
  CONSTRAINT ck_applicant_offer      CHECK (offer IS NULL OR jsonb_typeof(offer) = 'object'),
  CONSTRAINT ck_applicant_score      CHECK (overall_score IS NULL OR overall_score BETWEEN 1 AND 5),
  CONSTRAINT ck_applicant_hired      CHECK (converted_employee_id IS NULL OR stage = 'HIRED')
);
CREATE INDEX ix_applicant_stage ON tab_job_applicant (org_id, job_id, stage);

COMMIT;


-- ====================================================================
-- MODULE 07 : LIFECYCLE, ONBOARDING, OFFBOARDING & ASSETS
-- Alters : none (employee columns were added in Module 01)
-- New    : tab_asset  (+ FNF approval gate)
-- ====================================================================
BEGIN;

-- D14. tab_asset
CREATE TABLE tab_asset (
  asset_id             integer GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  org_id               integer      NOT NULL,
  app_id               integer      NOT NULL,
  asset_tag            varchar(50)  NOT NULL,
  category_code        varchar(30),
  asset_name           varchar(150) NOT NULL,
  brand_model          varchar(150),
  serial_number        varchar(100) NOT NULL,
  purchase_date        date,
  warranty_expiry_date date,
  status               varchar(12)  NOT NULL DEFAULT 'IN_STOCK',
  assigned_employee_id integer      REFERENCES tab_employee_master(employee_id),
  assigned_on          date,
  issue_condition      varchar(100),
  ack_at               timestamptz,
  expected_return      date,
  returned_on          date,
  return_condition     varchar(100),
  created_at           timestamptz  NOT NULL DEFAULT now(),
  updated_at           timestamptz  NOT NULL DEFAULT now(),
  CONSTRAINT uq_asset_serial UNIQUE (org_id, serial_number),
  CONSTRAINT ck_asset_status CHECK (status IN ('IN_STOCK','ASSIGNED','REPAIR','RETIRED')),
  CONSTRAINT ck_asset_return_condition CHECK (return_condition IS NULL OR return_condition IN
    ('RETURNED_IN_GOOD_CONDITION','DAMAGED','LOST')),
  CONSTRAINT ck_asset_custody CHECK (
    (status = 'ASSIGNED' AND assigned_employee_id IS NOT NULL AND assigned_on IS NOT NULL)
    OR (status <> 'ASSIGNED')),
  CONSTRAINT ck_asset_dates CHECK (returned_on IS NULL OR assigned_on IS NULL OR returned_on >= assigned_on)
);
CREATE UNIQUE INDEX uq_asset_tag ON tab_asset (org_id, app_id, asset_tag);
CREATE INDEX ix_asset_custodian ON tab_asset (org_id, assigned_employee_id);
CREATE INDEX ix_asset_warranty  ON tab_asset (warranty_expiry_date) WHERE warranty_expiry_date IS NOT NULL;

-- Issue and return history comes from the audit log
CREATE TRIGGER trg_asset_audit
  AFTER INSERT OR UPDATE ON tab_asset
  FOR EACH ROW EXECUTE FUNCTION fn_audit_row
    ('asset_id','assigned_employee_id','status','return_condition','returned_on');

-- An FNF run cannot be approved while an employee in it has an open blocking task
-- or an asset still ASSIGNED
```

```
CREATE OR REPLACE FUNCTION fn_fnf_gate() RETURNS trigger
LANGUAGE plpgsql AS $$
DECLARE v_emp integer;
BEGIN
  IF NEW.run_type = 'FNF' AND NEW.status = 'APPROVED' AND OLD.status IS DISTINCT FROM 'APPROVED' THEN
    SELECT i.employee_id INTO v_emp
    FROM (SELECT DISTINCT employee_id FROM tab_payroll_item_detail WHERE run_id = NEW.run_id) i
    WHERE EXISTS (SELECT 1 FROM tab_employee_record r
                  WHERE r.org_id = NEW.org_id AND r.employee_id = i.employee_id
                    AND r.is_blocking AND r.status IS DISTINCT FROM 'DONE')
       OR EXISTS (SELECT 1 FROM tab_asset a
                  WHERE a.org_id = NEW.org_id AND a.assigned_employee_id = i.employee_id
                    AND a.status = 'ASSIGNED')
    LIMIT 1;
    IF v_emp IS NOT NULL THEN
      RAISE EXCEPTION 'FNF run blocked: employee % has an open blocking task or an unreturned asset', v_emp;
    END IF;
  END IF;
  RETURN NEW;
END $$;

CREATE TRIGGER trg_payroll_run_fnf_gate
  BEFORE UPDATE OF status ON tab_payroll_run
  FOR EACH ROW EXECUTE FUNCTION fn_fnf_gate();
```

```
COMMIT;


-- ======================================================================
-- MODULE 08 : PERFORMANCE APPRAISAL & OKR/KPI
-- Alters : tab_employee_performance_review
-- New    : tab_employee_kpi_goal
-- ======================================================================
BEGIN;

ALTER TABLE tab_employee_performance_review
  ADD COLUMN IF NOT EXISTS cycle_id                   integer REFERENCES tab_common_master(master_id),
  ADD COLUMN IF NOT EXISTS reviewer_type              varchar(10),
  ADD COLUMN IF NOT EXISTS reviewer_employee_id       integer REFERENCES tab_employee_master(employee_id),
  ADD COLUMN IF NOT EXISTS competency_scores          jsonb,
  ADD COLUMN IF NOT EXISTS final_rating               numeric(3,1),
  ADD COLUMN IF NOT EXISTS promotion_recommended      boolean NOT NULL DEFAULT false,
  ADD COLUMN IF NOT EXISTS increment_recommended_pct  numeric(5,2),
  ADD COLUMN IF NOT EXISTS status                     varchar(20);
-- legacy uuid reviewer_id stays for old rows

ALTER TABLE tab_employee_performance_review
  ADD CONSTRAINT uq_review_rater UNIQUE (org_id, employee_id, cycle_id, reviewer_type, reviewer_employee_id),
  ADD CONSTRAINT ck_review_type CHECK (reviewer_type IS NULL OR reviewer_type IN
    ('SELF','MANAGER','PEER','HR','FINAL')),
  ADD CONSTRAINT ck_review_rating CHECK (final_rating IS NULL OR final_rating BETWEEN 1.0 AND 5.0),
  ADD CONSTRAINT ck_review_increment CHECK (increment_recommended_pct IS NULL OR increment_recommended_pct BETWEEN 0 AND 100),
  ADD CONSTRAINT ck_review_competency CHECK (competency_scores IS NULL OR jsonb_typeof(competency_scores) = 'object'),
  ADD CONSTRAINT ck_review_status CHECK (status IS NULL OR status IN
    ('DRAFT','SUBMITTED','CALIBRATED','FINAL')) NOT VALID;
CREATE INDEX IF NOT EXISTS ix_review_cycle ON tab_employee_performance_review (org_id, cycle_id, employee_id);

-- D15. tab_employee_kpi_goal
CREATE TABLE tab_employee_kpi_goal (
  goal_id            integer GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  org_id             integer       NOT NULL,
  app_id             integer       NOT NULL,
  employee_id        integer       NOT NULL REFERENCES tab_employee_master(employee_id),
  cycle_id           integer       REFERENCES tab_common_master(master_id),
  parent_goal_id     integer       REFERENCES tab_employee_kpi_goal(goal_id),
  goal_title         varchar(200)  NOT NULL,
  goal_type          varchar(20),
  unit               varchar(20),
  target_value       numeric(15,2),
  achieved_value     numeric(15,2),
  weightage_percent  numeric(5,2),
  progress_percent   numeric(5,2)  NOT NULL DEFAULT 0,
  due_date           date,
  manager_rating     numeric(3,1),
  status             varchar(12)   NOT NULL DEFAULT 'ACTIVE',
  fiscal_year_id     integer,
  created_at         timestamptz   NOT NULL DEFAULT now(),
  updated_at         timestamptz   NOT NULL DEFAULT now(),
  CONSTRAINT ck_goal_type     CHECK (goal_type IS NULL OR goal_type IN ('OBJECTIVE','KEY_RESULT','KPI')),
  CONSTRAINT ck_goal_status   CHECK (status IN ('ACTIVE','COMPLETED','ON_HOLD')),
  CONSTRAINT ck_goal_progress CHECK (progress_percent BETWEEN 0 AND 100),
  CONSTRAINT ck_goal_weight   CHECK (weightage_percent IS NULL OR weightage_percent BETWEEN 0 AND 100),
  CONSTRAINT ck_goal_rating   CHECK (manager_rating IS NULL OR manager_rating BETWEEN 1.0 AND 5.0),
  CONSTRAINT ck_goal_not_own_parent CHECK (parent_goal_id IS NULL OR parent_goal_id <> goal_id)
);
CREATE INDEX ix_goal_emp_cycle ON tab_employee_kpi_goal (org_id, employee_id, cycle_id);
```

```
-- Weightage of one employee's goals in a cycle should total 100: validate in the application.

COMMIT;


-- ======================================================================
-- MODULE 09 : LEARNING, TRAINING & CERTIFICATIONS
-- No new tables, no alters. Uses tab_common_master (HRMS_COURSE) and
-- tab_employee_record (TRAINING / CERTIFICATION) from Modules 00 and 01.
-- ======================================================================

-- ======================================================================
-- MODULE 10 : EXPENSE CLAIMS & TRAVEL REIMBURSEMENT
-- No new tables, no alters. Uses tab_hrms_request (EXPENSE_CLAIM) and
-- tab_payroll_item_detail (item_type REIMBURSEMENT, source EXPENSE).
-- ======================================================================


-- ======================================================================
-- MODULE 11 : HR HELPDESK, GRIEVANCES & TICKETING
-- New    : tab_helpdesk_ticket
-- ======================================================================
BEGIN;
```

```
-- D16. tab_helpdesk_ticket
CREATE TABLE tab_helpdesk_ticket (
  ticket_id           integer GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  org_id              integer      NOT NULL,
  app_id              integer      NOT NULL,
  ticket_no           varchar(30)  NOT NULL,
  employee_id         integer      NOT NULL REFERENCES tab_employee_master(employee_id),
  category_code       varchar(30),
  priority            varchar(10)  NOT NULL DEFAULT 'MEDIUM',
  subject             varchar(200) NOT NULL,
  description         text         NOT NULL,
  is_confidential     boolean      NOT NULL DEFAULT false,
  assigned_to         integer      REFERENCES tab_employee_master(employee_id),
  status              varchar(12)  NOT NULL DEFAULT 'OPEN',
  first_response_due  timestamptz,
  resolution_due      timestamptz,
  first_responded_at  timestamptz,
  resolved_at         timestamptz,
  resolution_notes    text,
  thread              jsonb        NOT NULL DEFAULT '[]'::jsonb,
  attachments         jsonb        NOT NULL DEFAULT '[]'::jsonb,
  created_at          timestamptz  NOT NULL DEFAULT now(),
  updated_at          timestamptz  NOT NULL DEFAULT now(),
  CONSTRAINT uq_ticket_no       UNIQUE (org_id, app_id, ticket_no),
  CONSTRAINT ck_ticket_priority CHECK (priority IN ('LOW','MEDIUM','HIGH','CRITICAL')),
  CONSTRAINT ck_ticket_status   CHECK (status IN ('OPEN','IN_PROGRESS','RESOLVED','CLOSED')),
  CONSTRAINT ck_ticket_thread   CHECK (jsonb_typeof(thread) = 'array'),
  CONSTRAINT ck_ticket_attach   CHECK (jsonb_typeof(attachments) = 'array'),
  CONSTRAINT ck_ticket_resolved CHECK (status NOT IN ('RESOLVED','CLOSED') OR resolved_at IS NOT NULL)
);
CREATE INDEX ix_ticket_emp    ON tab_helpdesk_ticket (org_id, employee_id, status);
CREATE INDEX ix_ticket_agent  ON tab_helpdesk_ticket (org_id, assigned_to, status);
CREATE INDEX ix_ticket_sla    ON tab_helpdesk_ticket (resolution_due) WHERE status IN ('OPEN','IN_PROGRESS');
-- Reply without overwriting: UPDATE tab_helpdesk_ticket SET thread = thread || $1::jsonb WHERE ticket_id = $2;

COMMIT;


-- ======================================================================
-- MODULE 12 : EMPLOYEE ENGAGEMENT, POLLS, SURVEYS & RECOGNITION
-- Alters : tab_announcements
-- New    : tab_engagement_entry
-- ======================================================================
BEGIN;

ALTER TABLE tab_announcements
  ADD COLUMN IF NOT EXISTS requires_ack  boolean     NOT NULL DEFAULT false,
  ADD COLUMN IF NOT EXISTS expires_at    timestamptz,
  ADD COLUMN IF NOT EXISTS target_scope  jsonb       NOT NULL DEFAULT '{}'::jsonb;
ALTER TABLE tab_announcements
  ADD CONSTRAINT ck_announcement_scope CHECK (jsonb_typeof(target_scope) = 'object');

-- D17. tab_engagement_entry
CREATE TABLE tab_engagement_entry (
  entry_id         bigint GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  org_id           integer      NOT NULL,
  app_id           integer      NOT NULL,
  entry_type       varchar(15)  NOT NULL,
  survey_id        integer      REFERENCES tab_common_master(master_id),
  from_employee_id integer      REFERENCES tab_employee_master(employee_id),
  to_employee_id   integer      REFERENCES tab_employee_master(employee_id),
```

```
  respondent_hash  varchar(64),
  rating           smallint,
  answers          jsonb,
  message          text,
  badge_code       varchar(30),
  created_at       timestamptz  NOT NULL DEFAULT now(),
  updated_at       timestamptz  NOT NULL DEFAULT now(),
  CONSTRAINT ck_engagement_type   CHECK (entry_type IN ('SURVEY_RESPONSE','POLL_VOTE','RECOGNITION')),
  CONSTRAINT ck_engagement_rating CHECK (rating IS NULL OR rating BETWEEN 1 AND 5),
  CONSTRAINT ck_engagement_answers CHECK (answers IS NULL OR jsonb_typeof(answers) IN ('object','array')),
  CONSTRAINT ck_engagement_recognition CHECK (
    entry_type <> 'RECOGNITION' OR (to_employee_id IS NOT NULL AND from_employee_id IS NOT NULL AND badge_code IS NOT NULL)),
  CONSTRAINT ck_engagement_survey CHECK (
    entry_type = 'RECOGNITION' OR survey_id IS NOT NULL),
  -- anonymous rows (from_employee_id NULL) must never link to a person
  CONSTRAINT ck_engagement_anonymous CHECK (
    from_employee_id IS NOT NULL OR (to_employee_id IS NULL AND entry_type <> 'RECOGNITION'))
);
CREATE UNIQUE INDEX uq_engagement_respondent ON tab_engagement_entry (org_id, survey_id, respondent_hash)
  WHERE respondent_hash IS NOT NULL;
CREATE INDEX ix_engagement_recognition ON tab_engagement_entry (org_id, to_employee_id)
  WHERE entry_type = 'RECOGNITION';
```

```
COMMIT;


-- ======================================================================
-- MODULE 13 : EXECUTIVE HR ANALYTICS  (10 views, no tables)
-- v_leave_balance is created in Module 03.
-- ======================================================================
BEGIN;

CREATE OR REPLACE VIEW v_headcount AS
SELECT e.org_id, e.app_id, e.branch_id, e.department_id, d.department_name, e.status,
       count(*) AS headcount,
       count(*) FILTER (WHERE e.join_date >= (current_date - interval '12 months')) AS joined_last_12m
FROM tab_employee_master e
LEFT JOIN tab_department_master d ON d.department_id = e.department_id
WHERE COALESCE(e.is_deleted, false) = false
GROUP BY e.org_id, e.app_id, e.branch_id, e.department_id, d.department_name, e.status;

CREATE OR REPLACE VIEW v_attrition AS
SELECT r.org_id, r.app_id,
       date_trunc('month', COALESCE(r.effective_date, r.decided_at::date))::date AS exit_month,
       r.request_type,
       COALESCE(r.payload->>'reason_code', r.payload->>'reason', 'UNSPECIFIED') AS exit_reason,
       count(*) AS exits
FROM tab_hrms_request r
WHERE r.request_type IN ('RESIGNATION','TERMINATION')
  AND r.status IN ('APPROVED','APPLIED')
GROUP BY r.org_id, r.app_id, 3, r.request_type, 5;

CREATE OR REPLACE VIEW v_absenteeism AS
SELECT a.org_id, a.app_id, e.department_id,
       date_trunc('month', a.attendance_date)::date AS month,
       count(*) FILTER (WHERE a.status = 'PRESENT')  AS present_days,
       count(*) FILTER (WHERE a.status = 'ABSENT')   AS absent_days,
       count(*) FILTER (WHERE a.status = 'LATE')     AS late_days,
       count(*) FILTER (WHERE a.status = 'HALF_DAY') AS half_days,
       count(*) FILTER (WHERE a.status = 'ON_LEAVE') AS leave_days,
       count(*) FILTER (WHERE a.status = 'WFH')      AS wfh_days,
       COALESCE(sum(a.late_minutes), 0)              AS late_minutes,
       COALESCE(sum(a.overtime_minutes), 0)          AS overtime_minutes,
       round(100.0 * count(*) FILTER (WHERE a.status = 'ABSENT')
             / NULLIF(count(*) FILTER (WHERE a.status NOT IN ('HOLIDAY','WEEK_OFF')), 0), 2) AS absenteeism_pct
FROM tab_employee_attendance a
JOIN tab_employee_master e ON e.employee_id = a.employee_id
GROUP BY a.org_id, a.app_id, e.department_id, 4;

CREATE OR REPLACE VIEW v_payroll_register AS
SELECT p.org_id, p.app_id, p.pay_period, p.run_id, e.department_id,
       count(*)                              AS employees,
       sum(p.gross_earnings)                 AS gross_earnings,
       sum(p.total_deductions)               AS total_deductions,
       sum(p.reimbursements)                 AS reimbursements,
       sum(p.net_pay)                        AS net_pay,
       sum(p.overtime_pay)                   AS overtime_cost,
       sum(p.employer_contribution)          AS employer_contribution,
       sum(p.gross_earnings + COALESCE(p.employer_contribution,0)) AS total_cost_to_company
FROM tab_employee_payroll p
JOIN tab_employee_master e ON e.employee_id = p.employee_id
WHERE p.run_id IS NOT NULL
GROUP BY p.org_id, p.app_id, p.pay_period, p.run_id, e.department_id;
CREATE OR REPLACE VIEW v_pf_ecr AS
```

```
SELECT i.org_id, i.app_id, i.pay_period, i.employee_id, e.full_name, s.uan_no, s.pf_no,
       max((i.rate_snapshot->>'wage')::numeric) FILTER (WHERE i.rate_snapshot->>'head' = 'PF_EE') AS pf_wages,
       COALESCE(sum(i.amount) FILTER (WHERE i.rate_snapshot->>'head' = 'PF_EE'),    0) AS employee_share,
       COALESCE(sum(i.amount) FILTER (WHERE i.rate_snapshot->>'head' = 'PF_EPF'),   0) AS employer_epf,
       COALESCE(sum(i.amount) FILTER (WHERE i.rate_snapshot->>'head' = 'PF_EPS'),   0) AS employer_eps,
       COALESCE(sum(i.amount) FILTER (WHERE i.rate_snapshot->>'head' = 'PF_EDLI'),  0) AS edli,
       COALESCE(sum(i.amount) FILTER (WHERE i.rate_snapshot->>'head' = 'PF_ADMIN'), 0) AS admin_charges
FROM tab_payroll_item_detail i
JOIN tab_employee_master e    ON e.employee_id = i.employee_id
JOIN tab_employee_statutory s ON s.employee_id = i.employee_id AND s.org_id = i.org_id
WHERE i.rate_snapshot->>'head' IN ('PF_EE','PF_EPF','PF_EPS','PF_EDLI','PF_ADMIN')
GROUP BY i.org_id, i.app_id, i.pay_period, i.employee_id, e.full_name, s.uan_no, s.pf_no;

CREATE OR REPLACE VIEW v_esi_return AS
SELECT i.org_id, i.app_id, i.pay_period, i.employee_id, e.full_name, s.esi_no,
       max((i.rate_snapshot->>'wage')::numeric) AS esi_wages,
       COALESCE(sum(i.amount) FILTER (WHERE i.rate_snapshot->>'head' = 'ESI_EE'), 0) AS employee_share,
       COALESCE(sum(i.amount) FILTER (WHERE i.rate_snapshot->>'head' = 'ESI_ER'), 0) AS employer_share
FROM tab_payroll_item_detail i
JOIN tab_employee_master e    ON e.employee_id = i.employee_id
JOIN tab_employee_statutory s ON s.employee_id = i.employee_id AND s.org_id = i.org_id
WHERE i.rate_snapshot->>'head' IN ('ESI_EE','ESI_ER')
GROUP BY i.org_id, i.app_id, i.pay_period, i.employee_id, e.full_name, s.esi_no;
```

```
CREATE OR REPLACE VIEW v_pt_return AS
SELECT i.org_id, i.app_id, i.pay_period, s.pt_state_code, i.employee_id, e.full_name,
       sum(i.amount) AS pt_amount
FROM tab_payroll_item_detail i
JOIN tab_employee_master e    ON e.employee_id = i.employee_id
JOIN tab_employee_statutory s ON s.employee_id = i.employee_id AND s.org_id = i.org_id
WHERE i.rate_snapshot->>'head' = 'PT'
GROUP BY i.org_id, i.app_id, i.pay_period, s.pt_state_code, i.employee_id, e.full_name;

-- PAN is encrypted: the Form 24Q export job decrypts it with fn_decrypt(), it is not exposed here
CREATE OR REPLACE VIEW v_form24q AS
SELECT i.org_id, i.app_id, i.pay_period, i.employee_id, e.full_name, s.tax_regime,
       sum(i.amount) AS tds_deducted
FROM tab_payroll_item_detail i
JOIN tab_employee_master e    ON e.employee_id = i.employee_id
JOIN tab_employee_statutory s ON s.employee_id = i.employee_id AND s.org_id = i.org_id
WHERE i.rate_snapshot->>'head' = 'TDS'
GROUP BY i.org_id, i.app_id, i.pay_period, i.employee_id, e.full_name, s.tax_regime;

-- Aggregates only: no employee identifiers are exposed
CREATE OR REPLACE VIEW v_diversity AS
SELECT e.org_id, e.app_id, e.department_id, e.gender,
       CASE
         WHEN e.date_of_birth IS NULL                                        THEN 'UNKNOWN'
         WHEN age(e.date_of_birth) < interval '25 years'                     THEN 'UNDER_25'
         WHEN age(e.date_of_birth) < interval '35 years'                     THEN '25_34'
         WHEN age(e.date_of_birth) < interval '45 years'                     THEN '35_44'
         WHEN age(e.date_of_birth) < interval '55 years'                     THEN '45_54'
         ELSE '55_PLUS'
       END AS age_band,
       count(*) AS headcount
FROM tab_employee_master e
WHERE COALESCE(e.is_deleted, false) = false
  AND e.status IN ('ACTIVE','PROBATION','NOTICE')
GROUP BY e.org_id, e.app_id, e.department_id, e.gender, 5;

COMMIT;


-- ============================================================================
-- FINAL : updated_at triggers, row-level security, grants, verification
-- ============================================================================
BEGIN;

-- updated_at trigger on every new table except append-only ones (audit log, leave ledger)
DO $$
DECLARE t text;
BEGIN
  FOREACH t IN ARRAY ARRAY[
    'tab_hrms_request','tab_hrms_approval_step','tab_employee_statutory','tab_employee_record',
    'tab_attendance_punch','tab_payroll_run','tab_employee_salary','tab_payroll_item_detail',
    'tab_job_posting','tab_job_applicant','tab_asset','tab_employee_kpi_goal',
    'tab_helpdesk_ticket','tab_engagement_entry'] LOOP
    EXECUTE format('CREATE TRIGGER trg_%s_updated BEFORE UPDATE ON %I FOR EACH ROW EXECUTE FUNCTION fn_set_updated_at()', t, t);
  END LOOP;
END $$;

-- Row-level security on org_id. The API must run, per request:
--   SELECT set_config('app.org_id', '<org from session>', true);
-- Note: table owners bypass RLS. Use a non-owner application role, or add FORCE ROW LEVEL SECURITY.
DO $$
```

```
DECLARE t text;
BEGIN
  FOREACH t IN ARRAY ARRAY[
    'tab_hrms_request','tab_hrms_approval_step','tab_hrms_audit_log','tab_employee_statutory',
    'tab_employee_record','tab_attendance_punch','tab_leave_ledger','tab_payroll_run',
    'tab_employee_salary','tab_payroll_item_detail','tab_job_posting','tab_job_applicant',
    'tab_asset','tab_employee_kpi_goal','tab_helpdesk_ticket','tab_engagement_entry'] LOOP
    EXECUTE format('ALTER TABLE %I ENABLE ROW LEVEL SECURITY', t);
    EXECUTE format($p$CREATE POLICY tenant_isolation ON %I
                     USING      (org_id = NULLIF(current_setting('app.org_id', true), '')::int)
                     WITH CHECK (org_id = NULLIF(current_setting('app.org_id', true), '')::int)$p$, t);
  END LOOP;
END $$;

-- Confidential tickets: requester, assigned agent and compliance officers only
DROP POLICY tenant_isolation ON tab_helpdesk_ticket;
CREATE POLICY ticket_visibility ON tab_helpdesk_ticket
  USING (
    org_id = NULLIF(current_setting('app.org_id', true), '')::int
    AND (
      NOT is_confidential
      OR current_setting('app.is_compliance', true) = 'true'
      OR employee_id  = NULLIF(current_setting('app.employee_id', true), '')::int
```

```
      OR assigned_to  = NULLIF(current_setting('app.employee_id', true), '')::int
    ))
  WITH CHECK (org_id = NULLIF(current_setting('app.org_id', true), '')::int);

-- Roles and grants for sensitive tables
DO $$
BEGIN
  IF NOT EXISTS (SELECT 1 FROM pg_roles WHERE rolname = 'hrms_hr')      THEN CREATE ROLE hrms_hr      NOLOGIN; END IF;
  IF NOT EXISTS (SELECT 1 FROM pg_roles WHERE rolname = 'hrms_payroll') THEN CREATE ROLE hrms_payroll NOLOGIN; END IF;
  IF NOT EXISTS (SELECT 1 FROM pg_roles WHERE rolname = 'hrms_recruiter') THEN CREATE ROLE hrms_recruiter NOLOGIN; END IF;
END $$;

REVOKE ALL ON tab_employee_statutory, tab_employee_salary, tab_payroll_item_detail,
              tab_payroll_run, tab_job_applicant FROM PUBLIC;
GRANT SELECT, INSERT, UPDATE ON tab_employee_statutory TO hrms_hr, hrms_payroll;
GRANT SELECT, INSERT, UPDATE ON tab_employee_salary, tab_payroll_item_detail, tab_payroll_run TO hrms_hr, hrms_payroll;
GRANT SELECT, INSERT, UPDATE ON tab_job_applicant TO hrms_hr, hrms_recruiter;

COMMIT;

-- ---------------------------------------------------------------------
-- VERIFICATION (should return 16 rows, then 10 rows)
-- ---------------------------------------------------------------------
/*
SELECT relname FROM pg_class
WHERE relkind IN ('r','p') AND relispartition = false AND relnamespace = 'public'::regnamespace
  AND relname IN ('tab_hrms_request','tab_hrms_approval_step','tab_hrms_audit_log','tab_employee_statutory',
    'tab_employee_record','tab_attendance_punch','tab_leave_ledger','tab_payroll_run','tab_employee_salary',
    'tab_payroll_item_detail','tab_job_posting','tab_job_applicant','tab_asset','tab_employee_kpi_goal',
    'tab_helpdesk_ticket','tab_engagement_entry')
ORDER BY 1;

SELECT viewname FROM pg_views WHERE schemaname='public' AND viewname IN
 ('v_headcount','v_attrition','v_absenteeism','v_leave_balance','v_payroll_register',
  'v_pf_ecr','v_esi_return','v_pt_return','v_form24q','v_diversity');
*/
```